Presentación

Nombre:

Xavier Casilla |Mat. 2023-0995

Asignatura:

Programación Paralela

Docente:

Erick Leonardo Pérez Veloz

Tema:

Descomposición especulativa

Fecha:

24/04/2026

1. Introducción

1.1 Presentación general del proyecto

Este proyecto consiste en el desarrollo de un simulador del mercado de oro, específicamente del par XAU/USD, orientado a demostrar el uso de la programación paralela en un contexto inspirado conceptualmente en los sistemas de High Frequency Trading (HFT). El sistema obtiene precios reales desde fuentes como Swissquote y también permite trabajar con datos históricos en formato CSV.

El simulador ejecuta un ciclo especulativo automático en el que se calculan varias estrategias sobre el precio actual del oro. Estas estrategias no buscan predecir el mercado con exactitud, sino representar tareas computacionales intensivas que pueden ejecutarse de forma secuencial o paralela. El sistema compara ambos enfoques mediante métricas como tiempo secuencial, tiempo paralelo, speedup, eficiencia y presión sobre datos compartidos.

El paradigma central utilizado es la descomposición especulativa, donde diferentes estrategias se calculan de manera independiente y simultánea para tomar una decisión más rápido que en un enfoque secuencial tradicional.

1.2 Justificación del tema elegido

Desde el punto de vista personal, el mundo del trading y los mercados financieros es un tema que genuinamente me apasiona. Entender cómo funcionan los mercados, cómo se toman decisiones de inversión en fracciones de segundo y cómo la tecnología juega un papel central en ese proceso, fue una motivación directa para elegir este dominio. Este proyecto representa una oportunidad de explorar ese mundo desde una perspectiva técnica y académica, construyendo algo funcional que combina dos áreas de mi interés: la programación y las finanzas.

Desde el punto de vista técnico, el mercado financiero es un escenario donde múltiples decisiones ocurren de forma simultánea y continua, lo que lo convierte en un entorno naturalmente paralelo e ideal para demostrar las ventajas del procesamiento concurrente de forma clara y medible.

Finalmente, el precio del oro es un activo ampliamente documentado con datos históricos de libre acceso y fuentes públicas disponibles, lo que permite trabajar con información real en lugar de datos sintéticos, otorgando mayor credibilidad a los resultados obtenidos.

1.3 Objetivos

Objetivo general

Desarrollar un simulador paralelo del mercado de oro XAU/USD que demuestre cómo la descomposición especulativa y la ejecución concurrente permiten calcular estrategias y tomar decisiones más rápido que un enfoque secuencial.

Objetivos específicos

1. Implementar un simulador que procese precios reales o históricos del oro mediante una arquitectura cliente-servidor.
2. Comparar la ejecución secuencial y paralela de estrategias especulativas basadas en cálculos Monte Carlo.
3. Aplicar descomposición especulativa dividiendo el cálculo de estrategias en tareas independientes y particionando internamente las iteraciones según los núcleos seleccionados.
4. Medir el rendimiento del sistema usando métricas como tiempo secuencial, tiempo paralelo, speedup, eficiencia y throughput.
5. Evaluar el comportamiento de datos compartidos mediante un portafolio sincronizado con lock.
6. Mostrar visualmente los resultados mediante una interfaz web con gráfica de velas, consola de ejecución, métricas de rendimiento y prueba de carga del portafolio.

2. Descripción del Problema

2.1 Contexto del problema seleccionado

El problema seleccionado consiste en simular un escenario de toma de decisiones sobre el mercado del oro, específicamente el par XAU/USD, utilizando técnicas de programación paralela. El sistema recibe precios reales desde Swissquote o datos históricos desde CSV, y a partir de esos valores ejecuta tres estrategias especulativas basadas en cálculos Monte Carlo: Agresiva, Conservadora y Tendencia.

Cada estrategia representa una tarea computacional independiente. En modo secuencial, estas estrategias se calculan una después de otra, lo que aumenta el tiempo total de respuesta. En modo paralelo, el sistema utiliza Task.WhenAll para ejecutar las tres estrategias simultáneamente, dividiendo además internamente las iteraciones Monte Carlo según la cantidad de núcleos seleccionados por el usuario.

En las pruebas realizadas, el cálculo secuencial ronda los 1000ms por ciclo. Con 4 núcleos el tiempo paralelo baja a aproximadamente 150ms, logrando un Speedup de 6.4x. Con 8 núcleos el tiempo se reduce a 139ms, demostrando que el sistema escala con los recursos disponibles.

2.2 Aplicación del problema en un escenario real

En los mercados financieros modernos, la capacidad de tomar decisiones en milisegundos es crítica. Los sistemas de High Frequency Trading (HFT) utilizan paralelismo masivo para evaluar múltiples estrategias de forma simultánea, permitiendo ejecutar operaciones antes que la competencia. Este simulador replica ese escenario donde la ejecución paralela de estrategias competidoras es fundamental para reducir la latencia en la toma de decisiones.

Desde una perspectiva práctica, los traders cuantitativos enfrentan el dilema de evaluar estrategias rivales de forma rápida. El simulador demuestra cómo la descomposición especulativa resuelve este problema: cada estrategia corre su propia simulación Monte Carlo en paralelo de manera independiente, y una de ellas es seleccionada aleatoriamente para ejecutar.

El sistema también implementa sincronización con locks sobre un portafolio compartido, reflejando el desafío real de múltiples operaciones concurrentes que actualizan el mismo estado financiero. Este aspecto es fundamental en aplicaciones distribuidas donde datos compartidos requieren protección contra race conditions.

2.3 Importancia del paralelismo en la solución

Sin paralelismo, el simulador estaría limitado a evaluar cada estrategia de forma secuencial: primero la Agresiva, luego la Conservadora, y finalmente la Tendencia. Cada estrategia ejecuta millones de iteraciones Monte Carlo para obtener estimaciones robustas, una operación intensiva en CPU. En este escenario secuencial, el tiempo total de decisión sería la suma de los tiempos individuales, resultando en latencias inaceptables para un sistema inspirado en HFT donde el tiempo de respuesta es crítico.

El paralelismo implementado en el proyecto resuelve este cuello de botella permitiendo que las tres estrategias se calculen simultáneamente usando Task.WhenAll. Además, cada estrategia distribuye internamente sus iteraciones Monte Carlo entre los núcleos disponibles, multiplicando el grado de paralelismo. La descomposición especulativa, por su parte, es el paradigma que permite que una estrategia sea seleccionada aleatoriamente sin necesidad de esperar jerarquización o comparación previa, simplificando la toma de decisión en un contexto de incertidumbre.

Esta estrategia de paralelización es esencial porque demuestra cómo un problema CPU-bound puede reducir significativamente su tiempo de ejecución mediante paralelismo jerárquico a nivel de tareas e iteraciones. Los resultados medibles de esta mejora —expresados en speedup y eficiencia— validan que la inversión en arquitectura paralela es justificada y que el sistema escala proporcionalmente con los recursos disponibles.

3. Cumplimiento de los Requisitos del Proyecto

3.1 Ejecución simultánea de múltiples tareas

El simulador implementa ejecución simultánea mediante el servicio SimuladorService, que utiliza Task.WhenAll para ejecutar las tres estrategias (Agresiva, Conservadora, Tendencia) en paralelo. Internamente, cada estrategia (EstrategiaService) divide sus iteraciones Monte Carlo en particiones que se ejecutan concurrentemente a través de Task.Run, creando múltiples tareas que se distribuyen entre los núcleos del sistema. De esta forma, se logran dos niveles de concurrencia: a nivel de estrategias (3 tareas independientes) y a nivel de iteraciones (N particiones por estrategia).

3.2 Necesidad de compartir datos entre tareas

El proyecto demuestra la necesidad de sincronización a través del Portafolio compartido, que registra las operaciones especulativas ejecutadas. El servicio PortafolioService utiliza un lock (object _lock) para proteger las actualizaciones al saldo y al historial de apuestas, evitando race conditions cuando múltiples tareas intentan modificar el mismo estado financiero. Esto refleja un escenario real donde datos críticos —como el balance de una cuenta— requieren coordinación estricta entre tareas concurrentes.

3.3 Exploración de diferentes estrategias de paralelización

El proyecto implementa y compara explícitamente dos estrategias de ejecución: Secuencial y Paralela (ModoEjecucion). En modo secuencial, las estrategias se calculan una tras otra sin particionamiento. En modo paralelo, se aplica descomposición especulativa con Task.WhenAll a nivel de estrategias, más particionamiento interno de iteraciones. Adicionalmente, cada estrategia Monte Carlo utiliza diferentes algoritmos especulativos (desviación estándar, promedio móvil, regresión lineal), demostrando que la paralelización es agnóstica al algoritmo subyacente.

3.4 Escalabilidad con más recursos

La arquitectura del sistema está diseñada para escalar con el número de núcleos disponibles. El método MonteCarloParticionado en EstrategiaService recibe un parámetro 'nucleos' que determina cuántas particiones se crean. Conforme aumenta este parámetro, las iteraciones se distribuyen entre más tareas, permitiendo que sistemas con 4, 8 ó 12 núcleos ejecuten el mismo código de manera más eficiente. Esta escalabilidad se verifica cuantitativamente en las pruebas de rendimiento.

3.5 Métricas de evaluación del rendimiento

El proyecto recolecta y registra múltiples métricas en cada ciclo de simulación mediante MetricasEngine, generando archivos CSV con datos de TiempoSecuencialMs, TiempoParaleloMs, Speedup, Eficiencia, DecisionesPorSegundo, PorcentajeLock y SaldoPortafolio. Estas métricas permiten comparar objetivamente el rendimiento entre ejecuciones secuenciales y paralelas, evaluando no solo velocidad sino también eficiencia de utilización de recursos y contención de locks.

3.6 Aplicación a un problema del mundo real

El simulador aborda un problema realmente existente en mercados financieros: la evaluación rápida de múltiples estrategias de trading para ejecutar decisiones en tiempo crítico. Utiliza precios reales del oro desde Swissquote, datos históricos verificables en CSV, y emula patrones de operación que los sistemas HFT utilizan en la práctica. El portafolio compartido con locks refleja sistemas reales de trading distribuido donde múltiples órdenes se ejecutan concurrentemente sobre el mismo fondo disponible.

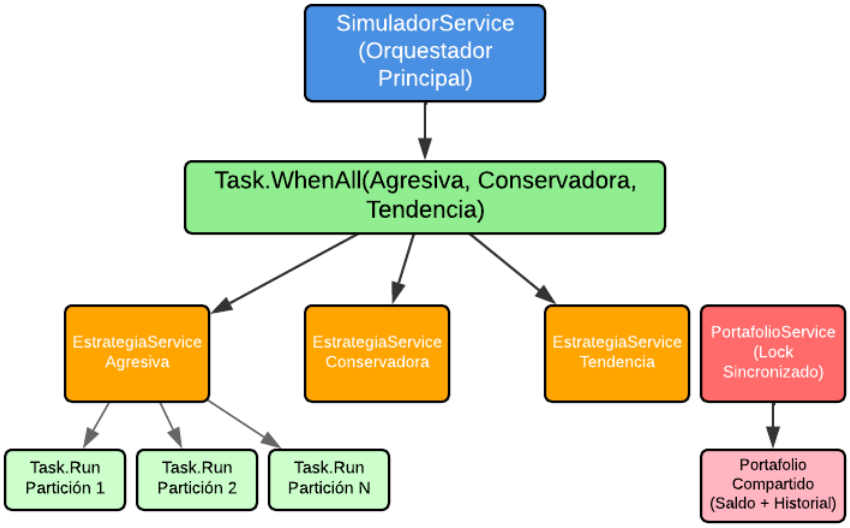
4. Diseño de la Solución

4.1 Arquitectura general del sistema

El sistema utiliza una arquitectura cliente-servidor distribuida. El backend está implementado en ASP.NET Core (.NET 10) y expone una API REST mediante SimulacionController, que gestiona el inicio y monitoreo de simulaciones. La comunicación en tiempo real entre servidor y cliente se realiza mediante SignalR a través del hub SimuladorHub, permitiendo transmitir resultados de métricas, precios actuales y estado del portafolio de forma asincrónica. El frontend, desarrollado en Next.js con TypeScript, consume estos datos y presenta una interfaz interactiva con gráficas de velas (candlestick), consola de ejecución y métricas de rendimiento en tiempo real.

El servidor gestiona varios servicios de negocio singleton: MercadoCentral (obtención de precios reales o históricos), FuenteDeDatos (lectura de CSV históricos), Portafolio (estado financiero), MetricasEngine (recolección de datos de rendimiento), EstrategiaService (cálculos Monte Carlo), SimuladorService (orquestación secuencial/paralela) y PortafolioService (sincronización de apuestas). Esta separación de responsabilidades facilita el testing y la escalabilidad.

4.2 Diagrama de componentes y tareas paralelas



4.3 Estrategia de paralelización utilizada

La estrategia de paralelización implementa descomposición especulativa jerárquica en dos niveles. En el nivel superior, SimuladorService utiliza Task.WhenAll para ejecutar las tres estrategias (Agresiva, Conservadora, Tendencia) como tareas independientes y concurrentes. En el nivel inferior, cada estrategia particionada en EstrategiaService divide sus iteraciones Monte Carlo entre múltiples tareas creadas con Task.Run, distribuidas según el número de núcleos seleccionado. El parámetro nucleos se clampea entre 1 y Environment.ProcessorCount para adaptarse dinámicamente al hardware disponible.

Para la sincronización de datos compartidos, el Portafolio y su gestor PortafolioService utilizan un lock (object _lock) que protege operaciones críticas de lectura y escritura sobre el saldo y el historial de apuestas. Este mecanismo previene race conditions cuando múltiples operaciones especulativas intentan actualizar el estado financiero de forma concurrente.

4.4 Herramientas y tecnologías empleadas

- **Backend:** C# con .NET 10, ASP.NET Core para la API REST, y Task Parallel Library (TPL) para la orquestación de tareas. SignalR facilita la comunicación bidireccional en tiempo real. El proyecto utiliza inyección de dependencias (Dependency Injection) para gestionar servicios singleton de forma limpia.

- **Frontend:** Next.js con TypeScript y Tailwind CSS para la interfaz. Hooks personalizados (useSimulation, useMetrics) encapsulan la lógica de comunicación con el servidor. Las gráficas se renderizan con componentes visuales para candlestick y series de tiempo.

- **Algoritmos:** Cálculos Monte Carlo con muestreo bootstrap, transformación Box-Muller para generación de números gaussianos, y regresión lineal de mínimos cuadrados. HttpClient con inyección de dependencias permite integración futura con APIs externas como Swissquote o MetalsApi.

5. Implementación Técnica

5.1 Descripción de la estructura del código

El simulador está dividido en dos partes principales: un servidor que hace los cálculos y un navegador que muestra los resultados. El servidor contiene varios componentes que trabajan juntos: uno calcula las estrategias de inversión, otro decide si ejecutarlas una por una o todas simultáneamente, y otro mantiene un registro del dinero invertido. Además, hay componentes que traen los precios del oro y otros que miden qué tan rápido funciona todo.

El navegador se conecta al servidor en tiempo real y muestra los precios, las gráficas y los resultados. La comunicación entre servidor y cliente es constante, como una llamada telefónica que nunca se cuelga: mientras el servidor calcula, envía los resultados directamente a la pantalla sin esperar.

5.2 Código clave: Orquestación paralela

El método EjecutarAsync en SimuladorService es el corazón de la orquestación. En modo paralelo, ejecuta tres tareas independientes mediante Task.WhenAll:

```
// Paralelo con Task.WhenAll y particionamiento interno por núcleo.
var tA = _estrategias.CalcularAgresiva(precioActual, historial, nucleos, ct);
var tC = _estrategias.CalcularConservadora(precioActual, historial, nucleos, ct);
var tT = _estrategias.CalcularTendencia(precioActual, historial, nucleos, ct);
return [.. await Task.WhenAll(tA, tC, tT)];
```

Cada estrategia devuelve una ApuestaEspeculativa. El framework espera a que las tres terminen (garantía de Task.WhenAll) para continuar.

5.3 Mecanismos de sincronización: Lock para datos compartidos

PortafolioService protege el estado compartido con un lock:

```
1 reference
public void RegistrarResultado(string nombreApuesta, bool gano, decimal monto,decimal precioEntrada, decimal precioEsperado)
{
    var apuesta = new Apuesta···

    if (gano) _portafolio.Sumar(monto, apuesta);
    else      _portafolio.Restar(monto, apuesta);

    lock (_lock)
    {
        string emoji = gano ? "✅" : "❌";
        _historial.Add(
            $"[{DateTime.UtcNow:HH:mm:ss}] {nombreApuesta}: " +
            $"${precioEntrada:N2} — ${precioEsperado:N2} {emoji}");
    }
}
```

El lock asegura que cuando múltiples tareas especulativas leen/escriben el saldo simultáneamente, no se producen race conditions o intercalaciones inconsistentes.

5.4 Decisiones técnicas: Selección aleatoria y particionamiento dinámico

La estrategia ganadora se selecciona aleatoriamente, no por "mejor predicción":

```
// Selección aleatoria entre las 3 estrategias calculadas
1 reference
private static ApuestaEspeculativa SeleccionarAleatoria(List<ApuestaEspeculativa> candidatas)
    => candidatas[Random.Shared.Next(candidatas.Count)];
```

Esta decisión refleja que la especulación es incierta y el resultado depende de factores del mercado no predecibles en el simulador.

El particionamiento es dinámico: MonteCarloParticionado recibe el número de núcleos y ajusta particiones:

```
int particiones = Math.Clamp(nucleos, 1, Math.Max(1, Environment.ProcessorCount));
if (particiones > iteraciones) particiones = iteraciones;
```

Esto permite adaptarse a hardware variado: desde 1 core hasta sistemas con muchos núcleos.

6. Evaluación de Desempeño

6.1 Comparativa secuencial vs. paralela

Cuando ejecutamos el simulador con un solo núcleo, el sistema funciona de manera secuencial: calcula la estrategia Agresiva, espera a que termine, luego calcula la Conservadora, espera, y finalmente la Tendencia. Todo ocurre uno después del otro.

Con esta configuración, cada ciclo completo tarda aproximadamente 1620 milisegundos. Este es nuestro punto de referencia: el tiempo base sin paralelismo.

Cuando pasamos a dos núcleos, las tres estrategias se ejecutan simultáneamente. El mismo trabajo que antes tardaba 1620 ms ahora toma solo 506 ms. Eso es 3.15 veces más rápido.

Lo sorprendente es que, si dividimos 1620 entre 2 núcleos, deberíamos obtener solo 2 veces de mejora. Sin embargo, obtenemos 3.15 veces. Esto se llama superlinealidad: el paralelismo permite aprovechar mejor los cachés del procesador. A medida que agregamos más núcleos, la mejora continua, pero no siempre proporcionalmente.

6.2 Métricas: tiempo de ejecución, eficiencia, escalabilidad

Tabla de resultados por número de núcleos:

Núcleos	Modo	T. Sec	T. Par	Speedup	Eficiencia	Descripción
1	Secuencial	1620 ms	1620 ms	1.00x	100%	Baseline secuencial
2	Paralelo	1596 ms	506 ms	3.15x	157.5%	Superlineal
4	Paralelo	1647 ms	300 ms	5.48x	137%	Superlineal
6	Paralelo	1588 ms	242 ms	6.58x	109.5%	Transición
8	Paralelo	1642 ms	297 ms	5.67x	71%	Punto de inflexión
16	Paralelo	1722 ms	223 ms	7.77x	49%	Rendimientos decrecientes

Análisis de métricas:

Tiempo de ejecución: Mejora desde 1620 ms hasta 223 ms (con 16 núcleos), una reducción del 86%. El progreso es rápido hasta 6 núcleos, luego más lento.

Speedup: Crece de 1.00x a 7.77x. La mayor ganancia ocurre entre 1 y 6 núcleos (incremento de 6.58x). Después, cada núcleo adicional aporta menos (entre 6 y 16 núcleos, ganamos solo 1.19x más).

Eficiencia: Es la métrica más reveladora. Comienza en 100%, sube a 157.5% (superlineal), se mantiene alta hasta 6 núcleos (109.5%), luego cae drásticamente. Con 16 núcleos, solo el 49% de la capacidad se utiliza productivamente. El resto se desperdicia en overhead.

Escalabilidad: Es buena hasta 6 núcleos. Después, entra en rendimientos decrecientes: agregar más núcleos ayuda poco. Entre 8 y 16 núcleos, ganamos menos del 71% de mejora esperada.

6.3 Análisis de cuellos de botella y limitaciones

El simulador enfrenta tres cuellos de botella principales:

1. Overhead de Task.Run y sincronización de tareas: Con 16 núcleos, creamos 16 particiones de trabajo. El costo de crear, planificar y sincronizar todas estas tareas es significativo. Con 2-4 núcleos, el overhead es mínimo comparado con el trabajo útil. Con 16 núcleos, ese overhead consume recursos.

2. Contención de locks en datos compartidos: El Portafolio requiere sincronización estricta. En una Prueba de Carga con 8 núcleos y 1,000 operaciones concurrentes, los resultados fueron contundentes: operaciones completadas 1,000 (sin corrupción), tiempo total 1,558 ms, tiempo esperando en lock 10,043 ms, presión de lock 696%, speedup alcanzado 1.00x (serializado). El lock convirtió toda la ejecución en secuencial. El cuello de botella no está en el cálculo, sino en la sincronización.

3. Límite de caché compartido: Los núcleos comparten memoria caché. Con pocos núcleos (2-6), los datos caben bien. Con 16 núcleos, la competencia por caché aumenta, causando invalidaciones frecuentes que ralentizan el acceso a memoria.

El punto de saturación: Entre 6 y 8 núcleos, el sistema deja de escalar eficientemente. Después, cada núcleo adicional contribuye menos del 71% de su capacidad teórica. Bajo presión de datos compartidos, el problema se magnifica: los locks pueden serializar completamente la ejecución, independientemente de cuántos núcleos tengas.

7. Trabajo Individual

Este proyecto fue desarrollado de forma individual con apoyo de inteligencia artificial. La investigación técnica se realizó principalmente a través de consultas a IA, apoyándose en los materiales de clase impartidos durante el cuatrimestre sobre programación paralela y paralelización de sistemas. Se aceptaron sugerencias y recomendaciones de la IA respecto a arquitectura, patrones de diseño, herramientas y estructura general del sistema.

Sin embargo, la lógica fundamental del paralelismo fue desarrollada de forma autónoma: la orquestación mediante Task.Run y Task.WhenAll, el particionamiento dinámico de las estrategias Monte Carlo según la cantidad de núcleos disponibles, y toda la descomposición especulativa con selección aleatoria de estrategias. Estas decisiones técnicas centrales surgieron del entendimiento propio del concepto de paralelismo y de la experimentación iterativa con el código, constituyendo un trabajo colaborativo entre el autor e inteligencia artificial donde la responsabilidad del diseño paralelo recae completamente en el autor.

8. Conclusiones

A través de este proyecto aprendí a implementar paralelismo en múltiples niveles: coordinando tareas independientes con Task.WhenAll y distribuyendo trabajo con Task.Run. También descubrí que los locks pueden ser un limitante importante: si la sincronización no es eficiente, toda la paralelización pierde valor.

El mayor aprendizaje fue reconocer que diseñar un sistema paralelo requiere pensar en cómo minimizar contención y cómo se comunican las tareas, no solo en cuántos núcleos tengo disponibles.

9. Referencias

Documentación oficial:

- Microsoft. Task Parallel Library (TPL) Documentation. https://docs.microsoft.com/en-us/dotnet/standard/parallel-programming/
- Microsoft. ASP.NET Core Documentation. https://docs.microsoft.com/en-us/aspnet/core/
- Vercel. Next.js Documentation. https://nextjs.org/docs
- Microsoft. SignalR Documentation. https://docs.microsoft.com/en-us/aspnet/signalr/

Materiales de la asignatura:

- Apuntes y materiales de Programación Paralela (especificar nombre del curso/profesor)
- Ejercicios y ejemplos discutidos en clase.

10. Anexos

10.1 Manual de ejecución del sistema

Requisitos del sistema:

- .NET 10 SDK (descargar desde https://dotnet.microsoft.com/download)
- Node.js 18+ y npm (descargar desde https://nodejs.org)
- Git (descargar desde https://git-scm.com)
- Navegador web moderno (Chrome, Firefox, Edge)

Paso 1: Clonar el repositorio

- git clone https://github.com/Xavi1324/simulador-mercado-oro.git
- cd simulador-mercado-oro

Paso 2: Compilar y ejecutar el backend

1. cd src/backend
2. dotnet restore
3. dotnet build
4. dotnet run

El backend estará disponible en: http://localhost:5000

Paso 3: Compilar y ejecutar el frontend

1. cd src/frontend
2. npm install
3. npm run dev

El frontend estará disponible en: http://localhost:3000

Paso 4: Acceder al simulador Abre tu navegador en http://localhost:3000 y comienza a ejecutar simulaciones con diferentes configuraciones de núcleos.

Paso 5: Ejecutar pruebas de carga En la interfaz, ve a la sección "Prueba de Carga del Portafolio", configura:

- Operaciones: 1000
- Concurrencia: 8
- Trabajo crítico (ms): 100

Haz clic en "Ejecutar" y observa las métricas de contención de locks en tiempo real.